

DATA ANALYTICS ASSESSMENT

SECTION A — CONCEPT APPLICATION

Question 1 — NumPy Vectorisation

Scenario: A delivery platform has 50,000 order distances and needs to calculate a delivery fee for every order.

Answer

NumPy vectorisation is technically superior to processing the 50,000 distances with a Python for-loop because the calculation can be performed on the complete NumPy array at once. NumPy operations are designed for numerical array processing and are generally faster and more efficient than repeatedly executing Python-level loop code. Vectorisation also makes the calculation shorter, clearer, and easier to maintain.

For example, if the delivery fee is Rs 5 per kilometre, a NumPy array called distances can be multiplied directly:

fees = distances * 5

There is no need to write a loop that takes one distance, calculates one fee, and appends it to a list. NumPy performs the operation across the array.

Broadcasting allows a scalar value to be used with an entire array. A scalar is a single value, such as 5. When 5 is multiplied by an array containing many distances, NumPy effectively applies the same value to every element of the array. For example, `[2, 5, 10] * 5` produces `[10, 25, 50]`.

A Python loop could still be necessary when every item requires a different, complicated decision or operation that cannot be conveniently expressed using NumPy array operations. For example, if each order had to call an external service separately or perform a custom step that depends on several changing conditions, a loop might be more appropriate.

Conclusion: For a simple numerical calculation applied to 50,000 values, NumPy vectorisation is the better choice because it is concise, efficient, and uses array operations directly.

Question 2 — Pandas groupby() and agg()

Scenario: A food delivery reporting pipeline needs average order value, delivery time, and rating for every restaurant.

Answer

Using groupby() with agg() is the correct approach because the task is a grouped summary. Instead of writing a for-loop for every restaurant, Pandas can divide the DataFrame into restaurant groups and calculate all required averages in one operation. This is shorter, easier to read, and less error-prone.

A suitable method chain is:

```
summary = (  
    df.groupby("restaurant_name")  
    .agg(  
        mean_order_value=("order_value", "mean"),  
        mean_delivery_time=("delivery_time_mins", "mean"),  
        mean_rating=("rating", "mean")  
    )  
)
```

The groupby("restaurant_name") part creates one group for each restaurant. The agg() part then calculates the three required averages for each group.

After this operation, restaurant_name becomes the DataFrame index by default. Each index value represents one restaurant, and the row contains that restaurant's calculated average order value, average delivery time, and average rating.

For example, if the original data contains orders from Restaurant A and Restaurant B, the resulting summary will have one row for Restaurant A and one row for Restaurant B. This makes the grouped result easy to filter, sort, and analyse.

Conclusion: groupby() + agg() is more suitable than manually looping through restaurants because the problem is naturally a grouped aggregation problem.

Question 3 — Choosing Matplotlib Charts

Scenario: A monthly report must show total orders per month, the proportion of orders by cuisine, and the distribution of delivery times.

Answer

(a) Total orders per month — Line chart

A line chart is appropriate because the data is measured across a sequence of months. The line makes it easy to see changes and trends over time, including increases, decreases, and peaks in order volume.

(b) Proportion of orders by cuisine — Pie chart

A pie chart is appropriate because the metric describes how the total number of orders is divided among the four cuisine categories: Fast Food, Indian, Chinese, and Desserts. Each slice represents a category's share of the whole.

(c) Distribution of delivery times — Histogram

A histogram is appropriate because delivery time is a numerical variable and the report needs to show how delivery times are distributed. The bins show how many orders fall within different delivery-time ranges.

The three charts can be arranged in a single figure using `plt.subplots()`. For example, three axes can be created in one row with:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```

Here, 1 means one row and 3 means three columns, so the three charts are placed side by side. Each chart can then be drawn on `axes[0]`, `axes[1]`, and `axes[2]`.

A shared x-axis would not be appropriate for this layout. The first chart has months on the x-axis, the pie chart does not use a normal numerical x-axis, and the histogram uses delivery-time ranges on its x-axis. Since the three charts represent different types of x-axis information, sharing one x-axis would not make sense.

Conclusion: the chart type should match the nature of the information: line for time trends, pie for proportions of a whole, and histogram for a numerical distribution.

Question 4 — Using plt.annotate()

Scenario: A 90-day line chart contains one highest-volume day, and stakeholders want that peak to be identified clearly.

Answer

First, I would find the position of the maximum order value in the array. With NumPy, `np.argmax()` can return the index of the largest value. The corresponding day number and order volume can then be used with `plt.annotate()`.

For example:

```
max_index = np.argmax(order_volume)
max_day = days[max_index]
max_value = order_volume[max_index]

plt.annotate(
    f"Day {max_day}: {max_value}",
    xy=(max_day, max_value),
    xytext=(max_day + 3, max_value + 200),
    arrowprops=dict(arrowstyle="->")
)
```

The `xy` parameter specifies the actual point being annotated. In this case, it points to the maximum-order day and its order volume.

The `xytext` parameter specifies where the annotation text should be placed. Moving the text away from the peak helps keep the label readable.

The `arrowprops` parameter controls the appearance of the arrow connecting the text to the point. For example, `arrowstyle="->"` creates an arrow pointing toward the peak.

This approach is programmatic because the maximum is found from the data itself. If the data changes, the program will still identify and annotate the new highest-volume day without manually finding the spike.

Question 5 — `sns.boxplot()` vs `sns.violinplot()`

Scenario: There are 10,000 customer ratings from three restaurant categories—Fast Food, Fine Dining, and Casual—with ratings on a discrete 1–5 scale.

Answer

For this particular dataset, I would recommend `sns.boxplot()`. A boxplot is a good choice because the manager wants to compare the distributions between three categories and identify extreme outliers.

First, there are 10,000 rows. With this many observations, a boxplot can give a compact summary of each category without displaying every individual observation. It shows the median, the middle 50% of values, and possible outliers in a simple form.

Second, there are only three categories, so the three boxplots can be compared easily side by side. The manager can quickly see which category has a higher or lower median and which category contains more extreme values.

The discrete 1–5 rating scale is another reason to prefer a boxplot here. Because only five rating values are possible, a violin plot's smooth density shape can be less informative and can give an impression of continuous data between the rating values. A boxplot provides a straightforward summary of the observed rating values and makes unusual observations easier to identify.

A violin plot would be preferable when the main goal is to understand the detailed shape of a distribution, especially with a continuous numerical variable and enough observations for the density shape to be meaningful. For example, if the dataset contained continuous delivery times for several restaurant categories, a violin plot could show differences in the shapes and concentrations of those distributions more clearly.

Conclusion: for 10,000 observations, three categories, and a discrete 1–5 rating scale where outliers are an important concern, the boxplot is the clearer and more direct choice.

Question 6 — Seaborn Correlation Heatmap and Multicollinearity

Scenario: A data science team wants to check eight numeric features before training a regression model.

Answer

I would first select the eight numeric columns and calculate their Pearson correlation matrix. The correlation matrix can then be displayed as a Seaborn heatmap. A simple approach is:

```
numeric_cols = [  
    "order_value",  
    "distance_km",  
    "delivery_time_mins",  
    "rating",  
    "discount_applied",  
    "peak_hour_flag",  
    "restaurant_age_years",  
    "driver_experience_years"  
]  
  
corr = df[numeric_cols].corr(method="pearson")  
sns.heatmap(corr, annot=True, cmap="coolwarm")
```

The heatmap colour represents the strength and direction of the Pearson correlation. A value close to +1 means a strong positive linear relationship, a value close to -1 means a strong negative linear relationship, and a value close to 0 means little or no linear relationship.

To read a specific cell, I would locate the row and column for two features. For example, if the cell for `distance_km` and `delivery_time_mins` contains 0.85, it means those two features have a strong positive linear relationship in the dataset: as distance tends to increase, delivery time also tends to increase.

For a practical multicollinearity check, I would flag an absolute correlation around 0.8 or higher as a potential warning sign. Therefore, correlations of about +0.8 or above, or -0.8 or below, deserve further investigation. The reason for using this type of threshold is that a high absolute correlation indicates that two predictors contain a large amount of similar linear information. In a regression model, strongly related predictors can make individual coefficient estimates less stable and harder to interpret.

This threshold should be treated as a screening rule rather than proof that multicollinearity definitely exists. A pair with a high correlation should be investigated further before deciding whether a feature should be removed or changed.

Conclusion: the heatmap gives a quick visual way to identify strongly related numeric features, while the actual correlation values provide the evidence needed to investigate possible multicollinearity.